

Assignment 1

Deep Machine Learning

Petter Eriksson

May 8, 2026

1 Question 1: Optimization Using Gradient Descent

The function used in this task is defined as:

$$f(x, y) = (x - 3)^2 + (y + 2)^2 \quad (1)$$

1.1 a) Partial Derivative with Respect to x

To compute the partial derivative with respect to x , y is treated as a constant.

$$\frac{\partial f}{\partial x} = \frac{\partial}{\partial x}(x - 3)^2 + \frac{\partial}{\partial x}(y + 2)^2$$

The first term depends on x , so using the chain rule:

$$\frac{\partial}{\partial x}(x - 3)^2 = 2(x - 3)$$

The second term depends only on y and is therefore constant with respect to x :

$$\frac{\partial}{\partial x}(y + 2)^2 = 0$$

Therefore:

$$\frac{\partial f}{\partial x} = 2(x - 3) \quad (2)$$

1.2 b) Partial Derivative with Respect to y

To compute the partial derivative with respect to y , x is treated as a constant.

$$\frac{\partial f}{\partial y} = \frac{\partial}{\partial y}(x - 3)^2 + \frac{\partial}{\partial y}(y + 2)^2$$

The first term depends only on x and is therefore constant with respect to y :

$$\frac{\partial}{\partial y}(x - 3)^2 = 0$$

The second term depends on y , so using the chain rule:

$$\frac{\partial}{\partial y}(y + 2)^2 = 2(y + 2)$$

Therefore:

$$\frac{\partial f}{\partial y} = 2(y + 2) \quad (3)$$

1.3 c) Gradient Descent Implementation

The gradient of the function is:

$$\nabla f(x, y) = [2(x - 3), 2(y + 2)] \quad (4)$$

Gradient descent updates the parameters by moving in the opposite direction of the gradient:

$$x_{t+1} = x_t - \eta \cdot 2(x_t - 3) \quad (5)$$

$$y_{t+1} = y_t - \eta \cdot 2(y_t + 2) \quad (6)$$

where η is the learning rate.

The implementation was written from scratch in Python. The starting point was chosen as:

$$x = 0, \quad y = 0$$

The learning rate was set to:

$$\eta = 0.1$$

and the algorithm was run for 50 iterations.

The final result from the implementation was approximately:

$$x \approx 2.999957 \quad (7)$$

$$y \approx -1.999971 \quad (8)$$

The final function value was approximately:

$$f(x, y) \approx 0$$

The exact global minimum is:

$$(x^*, y^*) = (3, -2) \quad (9)$$

At this point, the function reaches:

$$f(3, -2) = 0 \quad (10)$$

1.4 d) Gradient Descent Results

Loss Curve:

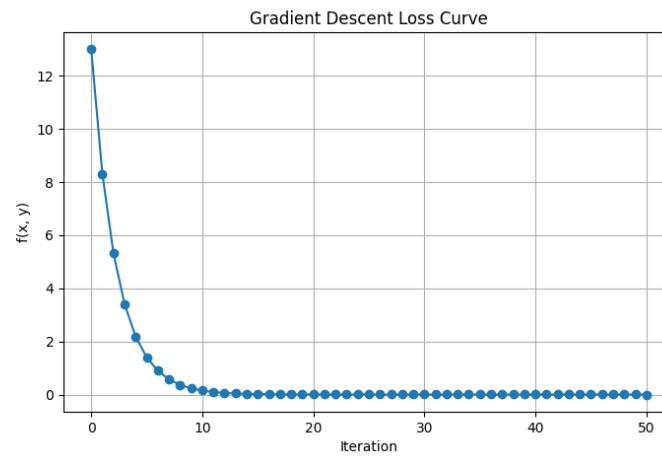


Figure 1: Loss during gradient descent

Optimization Trajectory:

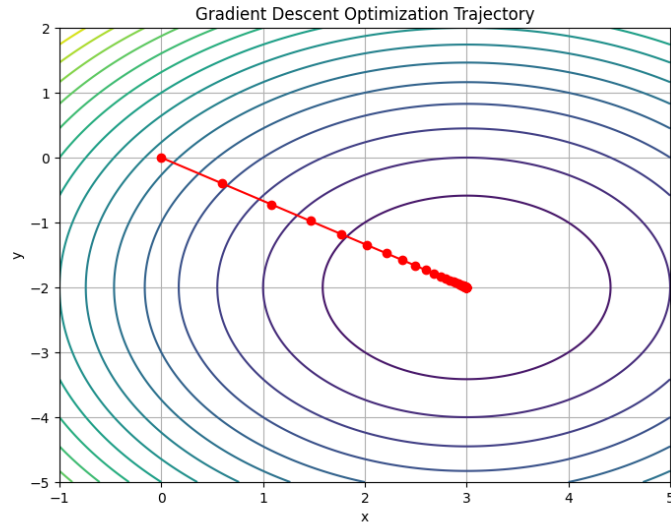


Figure 2: Optimization trajectory of (x, y)

Discussion:

The loss decreases monotonically over the iterations, which indicates stable convergence. Since the function is convex, gradient descent converges to the unique global minimum. The optimization trajectory also shows that the parameters move toward the minimum point $(3, -2)$.

Verification

The analytical derivatives were verified using *WolframAlpha*:

- $\frac{\partial}{\partial x}(x - 3)^2 + (y + 2)^2 = 2(x - 3)$
- $\frac{\partial}{\partial y}(x - 3)^2 + (y + 2)^2 = 2(y + 2)$

Source: WolframAlpha

2 Question 2: Deep Learning for Image Classification

This task focuses on image classification using the CIFAR-100 dataset. A custom convolutional neural network was designed and trained from scratch. The custom model was then compared against established architectures: ResNet-50, VGG-19, DenseNet-121, and EfficientNet.

2.1 1. Custom Architecture Description

The custom CNN consists of three convolutional blocks, each containing two convolutional layers, followed by fully connected layers. Each block includes convolution, batch normalization, ReLU activation, max pooling, and dropout.

- **Block 1:**

- Conv2d ($3 \rightarrow 32$), kernel 3×3 , stride 1, padding 1
- Conv2d ($32 \rightarrow 32$), kernel 3×3 , stride 1, padding 1
- Batch Normalization + ReLU after each convolution
- Max Pooling (2×2)
- Dropout ($p = 0.25$)

- **Block 2:**

- Conv2d ($32 \rightarrow 64$)
- Conv2d ($64 \rightarrow 64$)
- Batch Normalization + ReLU
- Max Pooling
- Dropout ($p = 0.25$)

- **Block 3:**

- Conv2d ($64 \rightarrow 128$)
- Conv2d ($128 \rightarrow 128$)
- Batch Normalization + ReLU
- Max Pooling
- Dropout ($p = 0.25$)

- **Fully Connected Layers:**

- Flatten
- Linear layer ($2048 \rightarrow 256$) + ReLU

- Dropout ($p = 0.5$)
- Output layer ($256 \rightarrow 100$)

The model contains six convolutional layers, satisfying the requirement of having between four and eight convolutional layers.

2.2 2. Training Configuration

The training configuration used for the custom CNN is shown below:

- Activation Function: ReLU
- Loss Function: CrossEntropyLoss
- Optimizer: Adam
- Learning Rate: 1×10^{-3}
- Batch Size: 128
- Custom CNN epochs: 100 with early stopping
- Pre-trained model epochs: 12 with early stopping
- Regularization:
 - Dropout with probability $p = 0.25$ in convolutional blocks and $p = 0.5$ in the fully connected classifier
 - Batch normalization after each convolutional layer
 - Weight decay of 1×10^{-4}

ReLU was used because it is computationally efficient and helps reduce the vanishing gradient problem. CrossEntropyLoss is suitable for multi-class classification problems such as CIFAR-100. Adam was selected because it adapts the learning rate for each parameter and usually provides stable convergence.

2.3 3. Data Preprocessing

The CIFAR-100 images are RGB images with a size of 32×32 pixels. The following preprocessing steps were applied:

- Normalization using CIFAR-100 mean and standard deviation:

$$\text{mean} = (0.5071, 0.4867, 0.4408)$$

$$\text{std} = (0.2675, 0.2565, 0.2761)$$

These values correspond to the standard normalization statistics of the CIFAR-100 training dataset.

- Data augmentation:
 - Random crop with padding
 - Random horizontal flip
 - Random rotation of 10 degrees
- Resizing was not required for the custom CNN because CIFAR-100 images are already 32×32 pixels.
- Grayscale conversion was not used because the color channels contain useful information for classification.

For the pre-defined architectures, resizing was applied when required because models such as ResNet-50, VGG-19, DenseNet-121, and EfficientNet are usually designed for larger input images.

2.4 4. Training Analysis

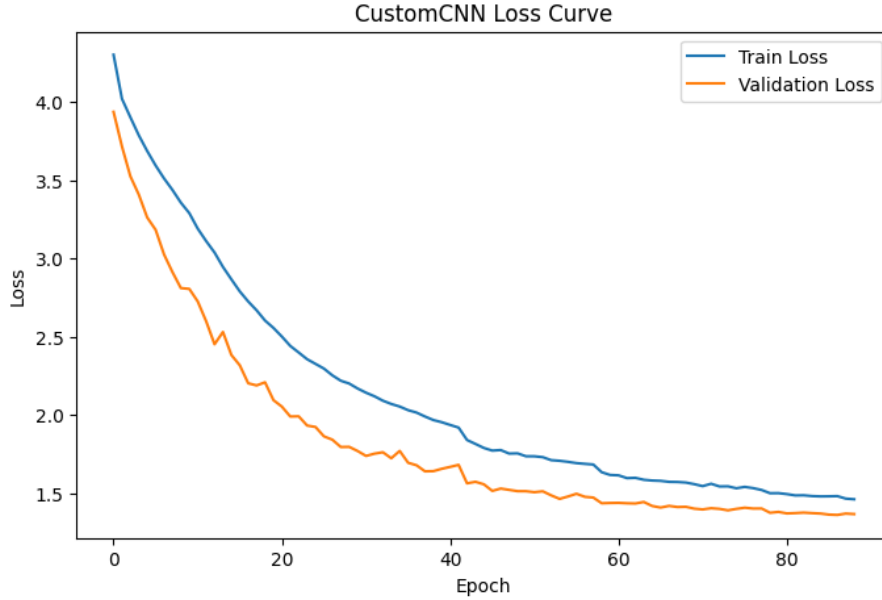


Figure 3: Training and validation loss for the Custom CNN

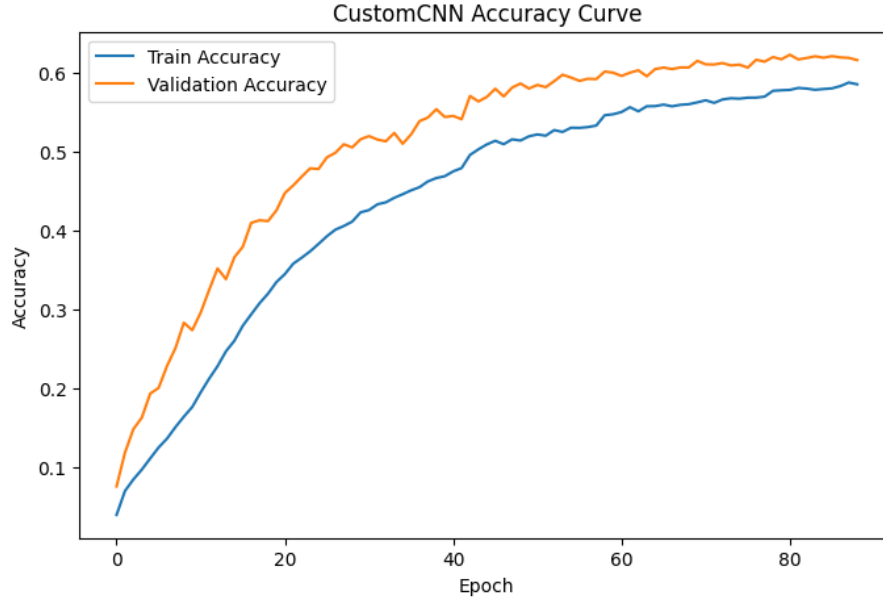


Figure 4: Training and validation accuracy for the Custom CNN

Discussion:

The training loss decreases over time, showing that the model learns from the training data. The validation loss also decreases during training, which indicates that the learned features generalize to unseen data.

If the training accuracy becomes significantly higher than the validation accuracy, this indicates overfitting. In this implementation, overfitting was reduced using dropout, batch normalization, weight decay, and data augmentation. These methods help the model generalize better instead of memorizing the training data.

2.5 5. Weight Evolution Analysis

To analyze how the network learns, one or two filters from the first convolutional layer were visualized before and after training.

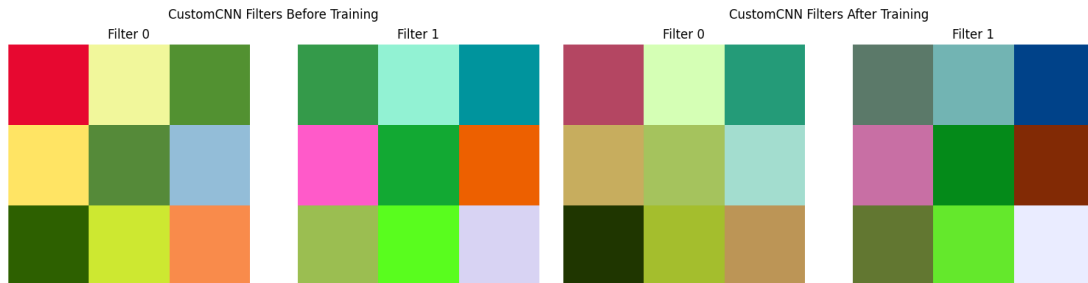


Figure 5: First-layer convolutional filters before and after training

Explanation:

Before training, the filters appear mostly random because the weights are initialized

without knowledge of the dataset. After training, the filters become more structured and start to capture useful visual patterns such as edges, color changes, and simple textures.

This is expected because early convolutional layers usually learn low-level features. These features are then combined by deeper layers to classify more complex objects.

2.6 6. Evaluation Metrics

The final model performance was evaluated using accuracy, true positives, true negatives, false positives, false negatives, and F1-score. The results are presented in Table 1.

For the multi-class setting, TP, TN, FP, and FN were computed by aggregating the one-vs-rest confusion matrix values over all 100 classes.

Model	Accuracy	TP	TN	FP	FN	F1-score
Custom CNN	0.6186	6186	986186	3814	3814	0.6145
ResNet-50	0.8589	8589	988589	1411	1411	0.8591
VGG-19	0.6834	6834	986834	3166	3166	0.6828
DenseNet-121	0.8370	8370	988370	1630	1630	0.8376
EfficientNet	0.8552	8552	988552	1448	1448	0.8555

Table 1: Model performance comparison on CIFAR-100 (best results in bold)

ResNet-50 achieves the best performance across all metrics, indicating the effectiveness of residual connections in training deep neural networks.

2.7 7. Model Comparison

The custom CNN achieved reasonable performance on CIFAR-100, but it is less advanced than the established architectures. The custom model has fewer layers and does not use advanced architectural ideas such as residual connections or dense feature reuse.

ResNet-50 performs well because its residual connections make it easier to train deep networks. These skip connections reduce the risk of vanishing gradients and allow the model to learn more complex feature representations.

VGG-19 is a deep architecture with many convolutional layers. It can learn strong image features, but it has many parameters and is computationally expensive compared to more modern architectures.

DenseNet-121 reuses features from earlier layers through dense connections. This improves gradient flow and can make the model more parameter-efficient.

EfficientNet uses compound scaling, balancing network depth, width, and input resolution. This often gives a strong trade-off between accuracy and computational cost.

Overall, the custom CNN is simpler and faster to train, but the pre-defined architectures are generally expected to achieve better classification performance because they are deeper and more carefully designed.

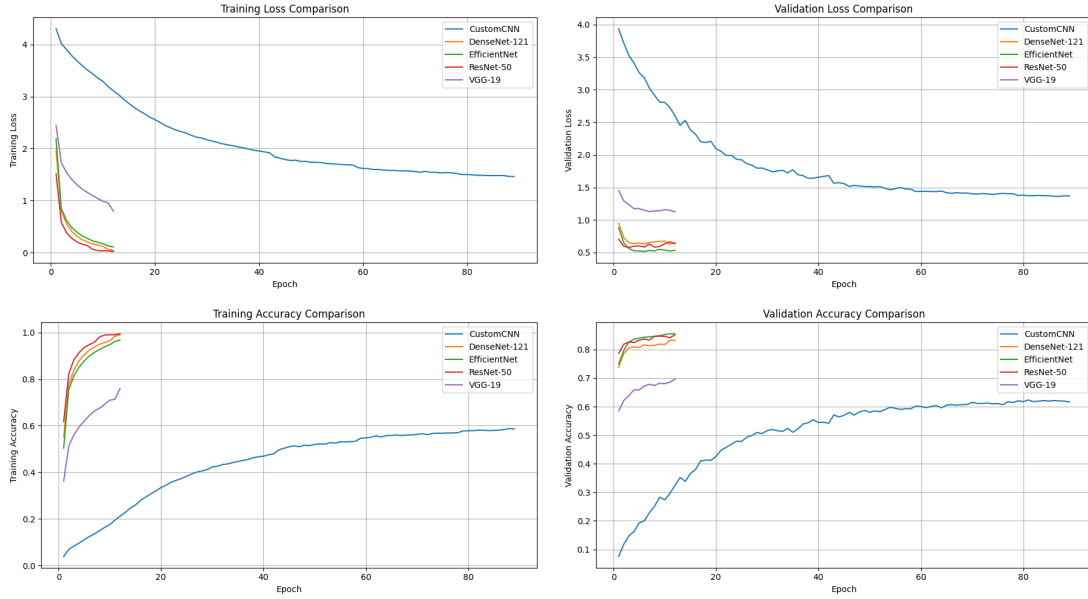


Figure 6: Comparison of training and validation loss and accuracy across all models

The comparison shows that deeper architectures generally achieve lower validation loss and higher validation accuracy compared to the custom CNN.

2.8 8. Reflection and Possible Improvements

The results show that the custom CNN performs reasonably well, but the predefined architectures generally achieve better performance due to their deeper structures and more advanced design choices, such as residual and dense connections.

One clear improvement would be to record epoch training times for each model. This would allow for a comparison not only of accuracy and loss, but also computational efficiency, which is an important factor when evaluating deep learning models in practice.

Further improvements could include tuning hyperparameters such as learning rate, dropout, and weight decay, as well as experimenting with different architectural choices. However, several of the models required significant training time, making it impractical to rerun multiple experiments within the available time. As a result, only a limited number of configurations were tested.

Future work should focus on more systematic hyperparameter tuning and a more balanced comparison that includes both performance and training efficiency.

References

- CIFAR-100 Dataset: <https://www.cs.toronto.edu/~kriz/cifar.html>
- A. Krizhevsky, "Learning Multiple Layers of Features from Tiny Images," 2009.

- Info and Model implementations adapted from ModelNova and PyTorch documentation.